

Syncaxis IAM - Architecture and Requirements

- [Syncaxis IAM — Centralized Identity & Access Control Module](#)
 - [0. What changed from v1, and why](#)
 - [1. What this is](#)
 - [2. Requirements](#)
 - [2.1 Functional](#)
 - [2.2 Non-functional](#)
 - [2.3 Assumptions \(flag if wrong — they change the design\)](#)
 - [3. High-Level Architecture](#)
 - [3.1 Component responsibilities](#)
 - [4. Authentication & SSO Flow](#)
 - [4.1 Direct login](#)
 - [4.2 SSO handoff \(already proven: Portal → Leads/ERP Dashboard today\)](#)
 - [4.3 Staying signed in: periodic re-verify, not shared tokens](#)
 - [4.4 Token lifetime](#)
 - [5. Why not the v1 shared-cookie design](#)
 - [6. Authorization Model](#)
 - [6.1 Permission key convention](#)
 - [6.2 Example — Leads Tracker](#)
 - [6.3 Role → Permission, User/Group → Role](#)
 - [6.4 Immediate revocation without a session table](#)
 - [6.5 What's deliberately out of scope \(for now\)](#)
 - [7. Data Model \(SQL Server — AuthCenter database\)](#)
 - [8. API Contract](#)
 - [8.1 Called by a consuming app's backend \(server-to-server only — never directly from a browser\)](#)
 - [8.2 Admin endpoints](#)
 - [8.3 App self-registration in practice](#)
 - [9. Deployment & Operations](#)
 - [9.1 Hosting](#)
 - [9.2 Network & TLS](#)
 - [9.3 Bootstrapping the first admin](#)
 - [9.4 Monitoring & backup](#)
 - [10. Admin UI](#)
 - [11. Technology Stack](#)
 - [12. Security Considerations](#)
 - [13. Migration & Rollout Plan](#)
 - [14. Mobile / Native Client Roadmap](#)
 - [14.1 What stays the same](#)
 - [14.2 Why the web pattern doesn't translate directly](#)
 - [14.3 The pattern to use: OAuth2 Authorization Code + PKCE](#)

- [14.4 Token lifetime for mobile — the one real schema addition, deferred until needed](#)
- [14.5 What Apps needs when this is built](#)
- [15. What to revisit as this grows](#)
- [16. Appendix: Implementation Specification](#)
 - [16.1 Language & package versions](#)
 - [16.2 Folder structure](#)
 - [16.3 Environment variables](#)
 - [16.4 API schemas](#)
 - [16.5 Validation rules](#)
 - [16.6 Bootstrap script \(`scripts/seed.ts` \)](#)
 - [16.7 Audit log conventions](#)

Syncaxis IAM — Centralized Identity & Access Control Module

Architecture & requirements document — v2 Author context: Syncaxis (Bosch Rexroth / Dobot distributor, India) · on-prem Windows Server + SQL Server Express (`SYNCAXIS-SERVER` , `192.168.3.9\SQLEXPRESS`) **Consumers at launch:** Company Portal, Leads Tracker, ERP Dashboard — designed to onboard any number of future apps without changes to this service

0. What changed from v1, and why

The first version of this document designed `syncaxis-iam` as a green-field service using a shared-parent-domain cookie for SSO. Since then, two things became clear from the actual codebases:

1. **Company Portal already implements a working identity system** — its own login, bcrypt+lockout, JWT issuance, role/group-based RBAC, and an admin UI (`UsersAdmin` , `RolesAdmin` , `UserGroupsAdmin`).
2. **Leads Tracker already integrates against it** using a pattern that is *not* the shared-cookie design from v1: it proxies login server-to-server, exchanges a one-time SSO code, keeps its own local session, and periodically re-verifies that session against Portal. No shared domain, no shared JWT secret exposed to the browser, no cookie-domain constraint. This is already live and working for both Leads Tracker and the ERP Dashboard.

This version keeps the goal of v1 — one central place that owns identity and access decisions for every app — but:

- **Extracts** that responsibility out of Portal into a dedicated `syncaxis-iam` service, instead of building a parallel one, so Portal stops being both an HR/company app and an identity provider.
- **Standardizes on the proven per-app-session pattern** (login-proxy / SSO-code-exchange / periodic re-verify) as the integration contract for every app, instead of a shared-cookie domain. This is strictly more flexible: apps can live on any host, port, or domain — a hard requirement for “open to future upcoming services.”
- **Replaces the access/refresh-token pair** from v1 with a single short-lived access token plus a per-user revocation cutoff (§6.4) — simpler to operate, and sufficient because no app ever reads this token directly; only `syncaxis-iam` itself ever validates it.

- **Deepens the permission model** from two flags (page / application) to per-action permission keys (`app.module.action`), so access control can express “can view this record type” vs. “can create/update/delete/export it,” not just “can open this page.”

1. What this is

A single, centrally-hosted **Identity & Access Management (IAM) service** — `syncaxis-iam` — that becomes the one place employees log in, and the one place that decides what they can see and do in every internal app. Every consuming app keeps its own session locally, but has no credentials, roles, or permissions of its own — it asks `syncaxis-iam` who a user is and what they’re allowed to do, once at login and periodically thereafter.

2. Requirements

2.1 Functional

- One login; a user signing in at Portal (or any app) can open any other app without re-entering credentials (SSO via code exchange, §5).
- Central store of users, roles, groups, and permissions in SQL Server — no other app stores credentials or access rules.
- **Fine-grained, per-action access control**, expressed per app as `<app>.<module>[.<subpage>].<action>` keys (§6) — e.g. `leads.leads.view`, `leads.leads.update`, `leads.leads.delete`, `erp.dashboard.finance.export`. Covers page/subpage visibility *and* the specific action (view/create/update/delete/export/...) within it.
- Apps **self-register** with `syncaxis-iam` — their identity (key, name, base URL) and the full list of permission keys they declare — without requiring any code change inside `syncaxis-iam` itself. This is what makes onboarding a future app additive rather than invasive.
- An admin UI to create users, manage roles/groups, and grant permissions per role, across every registered app, from one screen.
- Logout ends the local session in that app; an admin can force a user’s access to end everywhere within one re-verify interval (≤5 minutes) without waiting for token expiry.
- Audit trail of logins, permission denials, and admin changes.

2.2 Non-functional

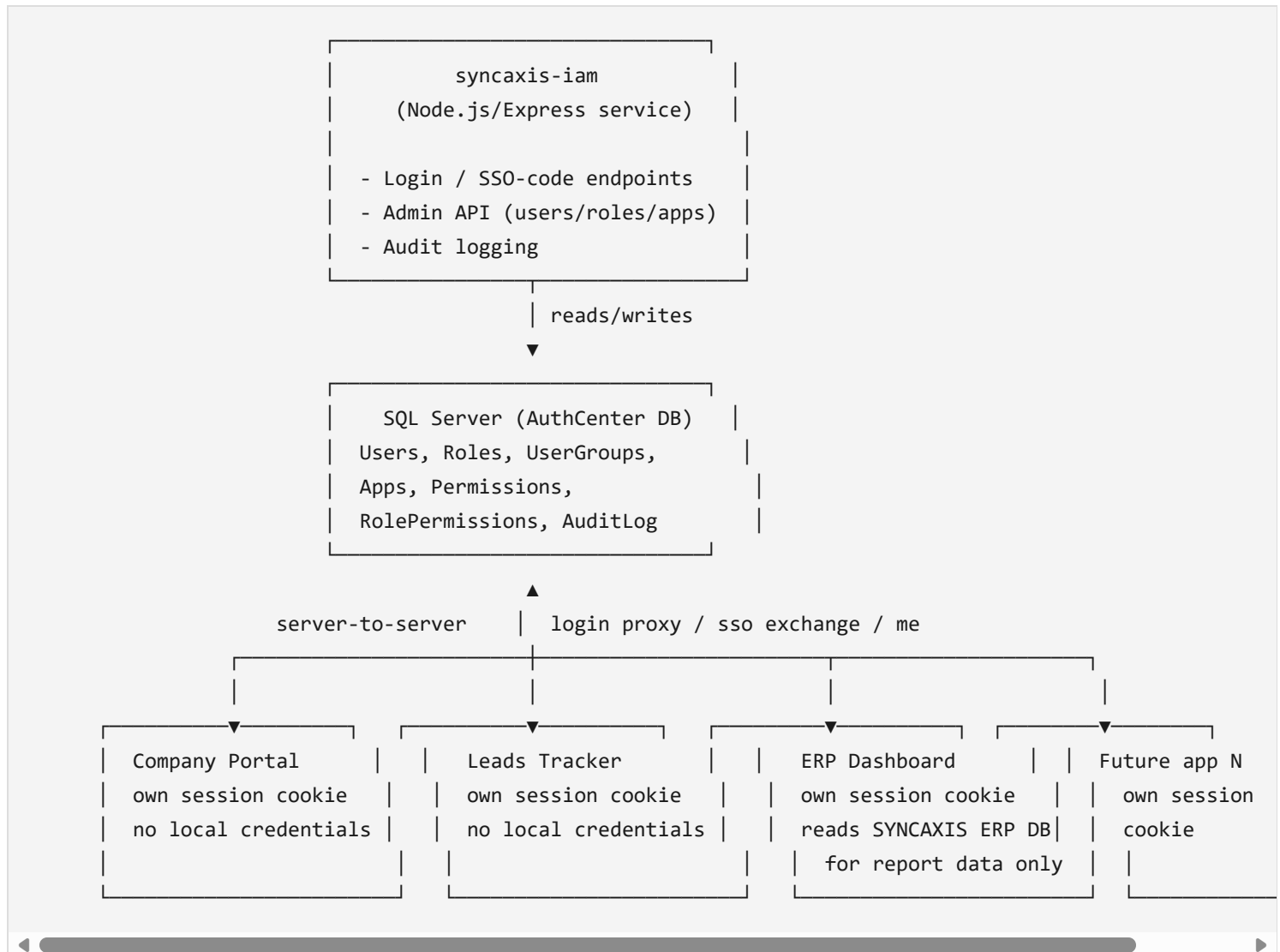
- **Scale:** tens to a few hundred employee accounts; a handful of apps today, headroom for many more. Optimize for simplicity and low ops burden over horizontal scalability.
- **Availability:** business-hours reliability; a single well-monitored instance with a documented restart procedure is adequate.
- **Latency:** LAN/VPN-speed internal traffic. A re-verify call to `syncaxis-iam` (at most once per 5 minutes per active user session) is not on the hot path of any request.
- **Cost:** self-hosted, no per-user licensing.
- **Extensibility (primary driver of this revision):** a new app onboards by (a) registering itself + its permission keys with `syncaxis-iam`, and (b) implementing the same three-endpoint client pattern every existing app already uses. No shared domain, no shared secret distributed to app configs, no change to `syncaxis-iam` code or schema.

- **Decoupling:** no consuming app depends on another consuming app. Every app depends only on `syncaxis-iam` being reachable; `syncaxis-iam` depends on nothing (it owns its own DB).

2.3 Assumptions (flag if wrong — they change the design)

- No existing Active Directory / LDAP — `AuthCenter` is the sole identity store. **Worth explicitly re-confirming**, since most Windows Server fleets do run AD; if you have it, §7 changes (federate/sync instead of hand-rolled passwords).
- Apps are Node/Express (or can add a small server-side client) and can make outbound HTTPS calls to `syncaxis-iam` — no requirement that they share a parent domain, cookie, or JWT secret with each other.
- HTTPS is achievable internally via a self-signed or internal CA certificate.

3. High-Level Architecture



Each app keeps its **own** local session (cookie + in-memory or DB-backed session record, its own TTL) — it never reads or trusts a token issued to another app, and never stores a password. `syncaxis-iam` is the only thing that ever checks a password or reads `AuthCenter`.

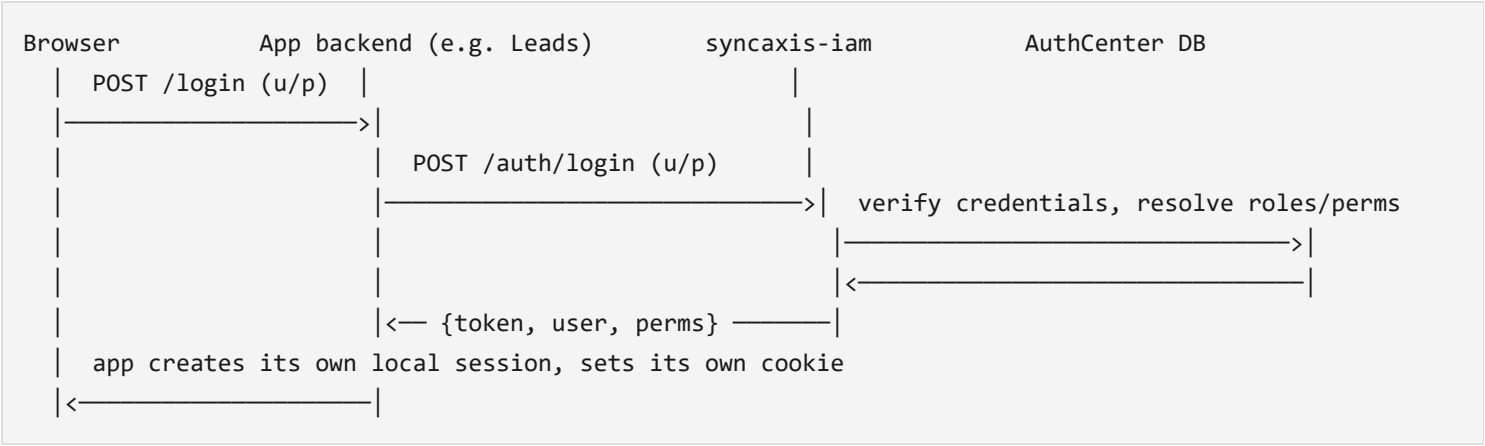
3.1 Component responsibilities

Component	Responsibility
syncaxis-iam service	Verify credentials, issue short-lived access tokens, exchange SSO codes, expose admin API, log every auth/access decision
AuthCenter SQL Server DB	Durable store for identity, roles, groups, permissions, audit log — touched only by syncaxis-iam
Each consuming app’s iam client (a small module inside the app, not a shared runtime)	Proxies login to syncaxis-iam , exchanges SSO codes, maintains a local session, re-verifies periodically, exposes req.session + hasPermission(key) to that app’s own routes
Each app	Owns its business logic/UI/DB entirely; never stores a credential or permission rule itself

4. Authentication & SSO Flow

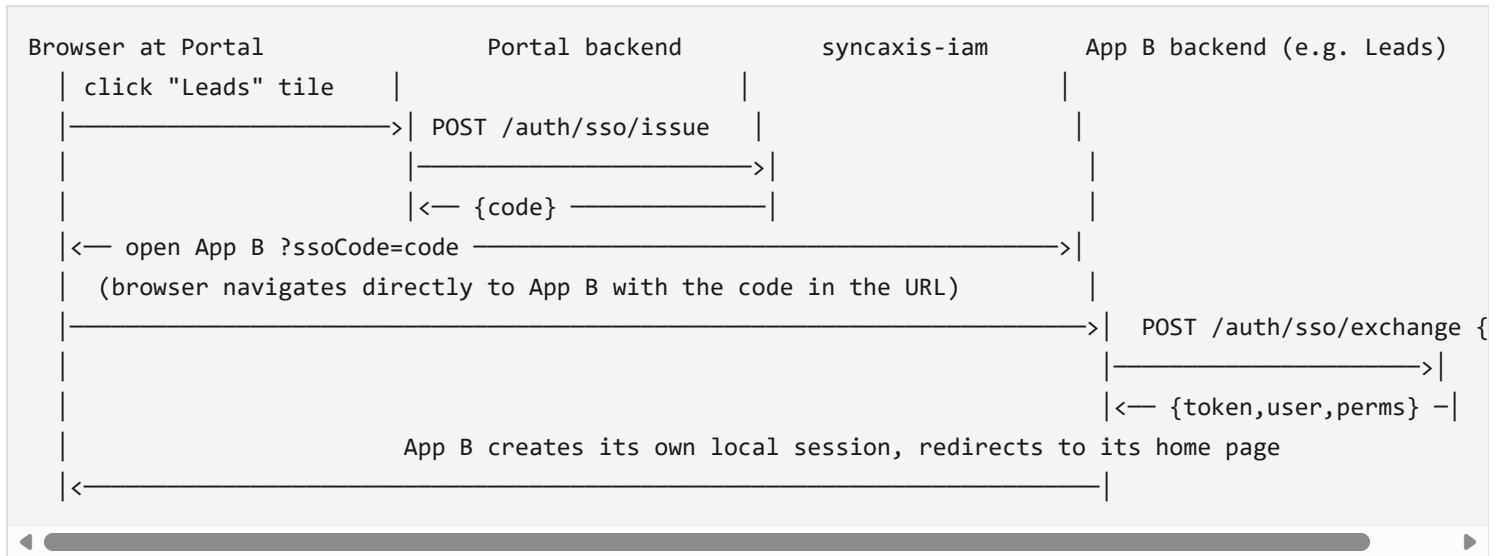
This is the pattern already proven by Leads Tracker against Portal — every current and future app uses the same three calls against syncaxis-iam instead.

4.1 Direct login



The app’s backend never stores the password; it only relays it once, server-to-server, and discards it.

4.2 SSO handoff (already proven: Portal → Leads/ERP Dashboard today)



The code is short-lived (60s) and single-use — it never reaches the browser as anything more than an opaque, already-expiring string, and is exchanged for a real session only server-to-server.

4.3 Staying signed in: periodic re-verify, not shared tokens

Each app's session record keeps the access token it received at login/SSO-exchange, server-side only (never sent to the browser). On each incoming request, if more than 5 minutes have passed since the last check, the app calls `GET /auth/me` on `syncaxis-iam` with that token:

- Still valid + user still active → session's cached permissions refresh, request proceeds.
- Token invalid, user deactivated, or the user's `TokenValidAfter` cutoff (§6.4) is newer than the token → session is destroyed, user is asked to sign in again.
- `syncaxis-iam` unreachable → keep the cached session until its hard expiry (matches the token's own expiry, default 8h) rather than logging everyone out over a network blip.

This gives near-real-time revocation (≤ 5 minutes) without a central session store, refresh-token rotation, or shared cookie — each app already fully implements this today (see Leads Tracker's `auth.ts`).

Trade-off to keep in mind: a permission or role change made in the admin UI doesn't reach an already-signed-in user until their next re-verify — up to 5 minutes, not instant. Acceptable for normal role changes; for anything urgent (e.g. offboarding), use `POST /admin/users/:id/force-logout` (§6.4, §8.2), which every app picks up on its very next re-verify regardless of the 5-minute cadence.

Logout is per-app, not global. Signing out of Portal does not sign a user out of Leads Tracker or the ERP Dashboard — each app's session is independent, matching what's already built today. An admin can end every session for a user immediately via force-logout (§6.4); there is currently no self-service “sign out everywhere” across apps. Revisit only if a concrete need for it shows up — it would reuse the same `TokenValidAfter` mechanism.

4.4 Token lifetime

Token	Held by	Lifetime	Notes
Access token	The consuming app’s backend only — never sent to the browser	8h (configurable)	Verified by <code>syncaxis-iam</code> on <code>/auth/me</code> ; not trusted or decoded by consuming apps
App session cookie	The user’s browser, scoped to that one app	Matches access token (8h)	<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code> ; independent per app — logging out of one doesn’t touch another

No refresh tokens, no rotation, no shared secret distributed to app configs — simpler than v1, and this is already what’s running today.

5. Why not the v1 shared-cookie design

	Shared-cookie SSO (v1)	Per-app session + code exchange (v2, this doc)
Domain requirement	All apps must share a parent domain (<code>.syncaxis.local</code>)	None — any host/port/domain
Secret distribution	JWT secret (or public key) must reach every app	Access tokens never leave <code>syncaxis-iam</code> ↔ app-backend; nothing to distribute
Adding a future app	Must be brought under the shared domain	Register + implement 3 endpoints, done
Already proven at Syncaxis	No	Yes — Leads Tracker ↔ Portal today

Given the explicit goal of staying loosely coupled and open to apps you haven’t built yet, this is the better fit — it’s also strictly less infrastructure to stand up (no internal DNS/subdomain plumbing required for SSO to work, though you may still want subdomains for tidiness).

6. Authorization Model

6.1 Permission key convention

```
<app>.<module>[.<subpage>].<action>
```

- `<app>` — the registering app’s key (`leads` , `erp` , `portal` , ...)
- `<module>` — a page or entity/resource type (`leads` , `customers` , `dashboard.finance` , ...)
- `<action>` — app-defined verb: `view` , `create` , `update` , `delete` , `export` , `approve` , ... — no fixed enum; each app declares whatever actions it needs.

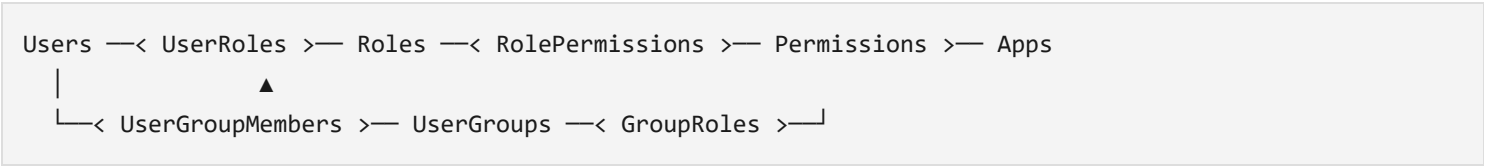
`syncaxis-iam` never interprets the string — it only needs it to be unique per app and mapped to roles. The app declaring the key is the one that decides what it gates.

6.2 Example — Leads Tracker

Key	Gates
<code>leads.leads.view</code>	See the Leads list/board
<code>leads.leads.create</code>	Add a lead
<code>leads.leads.update</code>	Edit a lead
<code>leads.leads.delete</code>	Delete a lead
<code>leads.leads.export</code>	Export to Excel
<code>leads.customers.view</code> / <code>.create</code> / <code>.update</code> / <code>.delete</code>	Same, for Customers
<code>leads.admin.manage</code>	Admin section of Leads Tracker itself

6.3 Role → Permission, User/Group → Role

Unchanged from v1’s relational model:



A user’s effective permission set is the union of every permission granted by every role assigned directly to them, or via any group they belong to (already implemented and working in Portal’s `getEffectiveAccess`). One role, `Admin` , is flagged `IsFullAccess` and bypasses all individual grants — the emergency/owner account, not a pattern to lean on for normal roles.

6.4 Immediate revocation without a session table

Rather than tracking every issued token (v1’s `RefreshTokens` table), `Users` carries a single `TokenValidAfter` timestamp. Any token issued *before* that timestamp is rejected by `/auth/me` . Bumping it — on password change, account deactivation, role change requiring an immediate cutoff, or an explicit “force logout” admin action — invalidates every session across every app within one re-verify cycle (≤5 minutes), with no per-session bookkeeping.

6.5 What’s deliberately out of scope (for now)

- **Row-level / record-level scoping** (“SalesRep sees only their own leads”) — not covered by this model; if needed later, it’s a filter each app applies in its own queries using the user’s id, independent of the permission-key system above. Not required for the current requirement.
- **Field-level masking** (hiding specific fields within a record per role) — same: out of scope until a concrete need appears.

Both can be layered on later without changing this document’s schema or API contract.

7. Data Model (SQL Server — AuthCenter database)

Deliberately separate from `SYNCAXIS` (ERP) and from Portal’s own DB — `syncaxis-iam` is the only thing that reads or writes it.

Table	Key columns	Notes
Users	UserId PK , Username / Email (unique), PasswordHash , DisplayName , IsActive , FailedLoginCount , IsLocked , TokenValidAfter , CreatedAt , LastLoginAt , PasswordChangedAt	bcrypt/argon2 hashes only
Roles	RoleId PK , Name (unique), Description , IsFullAccess	
UserRoles	UserId FK , RoleId FK	Composite PK
UserGroups	GroupId PK , Name , Description	Optional grouping layer (department, team, ...)
UserGroupMembers	UserId FK , GroupId FK	Composite PK
GroupRoles	GroupId FK , RoleId FK	A group grants its roles to every member
Apps	AppId PK , Key (unique), Name , BaseUrl , IsActive	Self-registered
Permissions	PermissionId PK , AppId FK , Key (unique per app), Description	Self-registered per app, per §6.1 convention
RolePermissions	RoleId FK , PermissionId FK	The access-control matrix
AuditLog	AuditId PK , UserId FK NULL , EventType , AppKey , Detail , IPAddress , CreatedAt	Login success/failure, SSO issue/exchange, permission denials, admin changes, force-logouts

Full `CREATE TABLE` statements are in the companion `db/02-syncaxis-iam-schema.sql` (run after `db/01-syncaxis-iam-create-db.sql` ; `db/03-syncaxis-iam-create-login.sql` creates the service’s least-privilege SQL login).

What deliberately isn’t here: business/profile data. `AuthCenter.Users` holds only what identity and access decisions need (credentials, roles, groups) — no employee record, department, photo, or job title. Portal’s *current* `portal.Users` table couples these together (it carries `EmployeeId` directly on the login row); after migration, that link has to live on Portal’s side instead — e.g. `portal.Employees.AuthUserId` referencing the `id` returned in `syncaxis-iam`’s `/auth/login` and `/auth/me` responses. Missing this during migration is the most likely way Portal’s Directory/org-chart features silently break — call it out explicitly in the migration step that removes `portal.Users` (§13, step 2).

8. API Contract

8.1 Called by a consuming app's backend (server-to-server only — never directly from a browser)

Method & path	Purpose
POST /auth/login	{username, password} → {token, user, perms} or an error the app relays as-is (invalid credentials, locked account)
POST /auth/sso/issue	Called by the <i>originating</i> app (e.g. Portal) once the user is already signed in there, to mint a one-time code for a tile/link to another app
POST /auth/sso/exchange	Called by the <i>destination</i> app with {code} → {token, user, perms} , single-use, 60s TTL
GET /auth/me	Bearer token → {user, perms} if still valid (checks IsActive , TokenValidAfter); 401 otherwise — this is what powers each app's periodic re-verify
POST /auth/logout	Optional — mainly for symmetry/audit; each app's own session destruction is what actually matters locally
GET /health	Liveness/readiness check (DB reachable) — for the monitoring in §9

8.2 Admin endpoints

Method & path	Purpose
POST /admin/apps	Register an app (key , name , baseUrl)
POST /admin/apps/:key/permissions	Bulk-declare/update an app's permission keys (idempotent — an app can call this on every startup)
GET/POST /admin/users	List/create users
POST /admin/users/:id/roles	Assign/remove roles
POST /admin/users/:id/groups	Assign/remove group membership
POST /admin/users/:id/force-logout	Bumps TokenValidAfter to now
GET/POST /admin/roles	List/create roles
POST /admin/roles/:id/permissions	Assign/remove permissions on a role
GET /admin/audit	Query the audit log

All admin endpoints require `iam.admin.manage` , granted like any other permission — `syncaxis-iam` treats its own admin console as just another registered app, same as v1.

8.3 App self-registration in practice

Each app keeps a small static manifest in its own code, e.g. `leads/src/permissions.ts` :

```
export const LEADS_PERMISSIONS = [
  { key: 'leads.leads.view', description: 'View leads list/board' },
  { key: 'leads.leads.create', description: 'Add a lead' },
  { key: 'leads.leads.update', description: 'Edit a lead' },
  { key: 'leads.leads.delete', description: 'Delete a lead' },
  { key: 'leads.leads.export', description: 'Export leads to Excel' },
  { key: 'leads.customers.view', description: 'View customers' },
  // ...
]
```

and calls `POST /admin/apps/:key/permissions` with it once at startup (or via a one-off script). New permission keys appear in the admin matrix automatically; nothing in `syncaxis-iam` needs to change.

Namespace enforcement: `syncaxis-iam` should reject any key that doesn't start with `<callingApp's own Key>`. — e.g. the `leads` app registering `erp.hijack.view` is rejected. Grants are already scoped correctly via `Permissions.AppId` regardless, but without this check a misbehaving or buggy app could register a confusingly-named key that shows up under the wrong app in the admin matrix. Cheap to add, not in the v1 schema/API — add it to the `POST /admin/apps/:key/permissions` handler, not the DB schema.

9. Deployment & Operations

9.1 Hosting

Same server that hosts `SYNCAXIS-SERVER` can run `syncaxis-iam` — it's a lightweight service — or a separate small VM if you'd rather isolate identity from the ERP box. Either **IIS + iisnode** (if you already run IIS, gets you auto-restart, log rotation, and TLS termination for free) or **PM2 as a Windows Service + nginx/IIS in front** works at this scale; pick whichever you're already comfortable operating. `pm2-windows-startup` or `node-windows` makes PM2/Node survive a reboot as a proper Windows service.

9.2 Network & TLS

No shared-parent-domain requirement (§5), but internal DNS entries (or `hosts` file entries) for `iam.syncaxis.local` and each app's own subdomain are still worth having for a stable, memorable URL each app's config points at. A self-signed cert or a minimal internal CA (via `certutil` / OpenSSL, root pushed to company machines via Group Policy if you have AD, or manually otherwise) is sufficient since this never leaves the LAN/VPN — but HTTPS is not optional even internally, since credentials cross this boundary on every login.

9.3 Bootstrapping the first admin

The admin API (§8.2) itself requires `iam.admin.manage`, which nothing can grant through the API before a first admin exists — a deliberate chicken-and-egg to avoid an unauthenticated “create the first admin” endpoint sitting in production code. Resolve it the same way Portal already does (`server/scripts/seed.js`): an idempotent `npm run seed` script, **not part of normal app startup**, driven by `SEED_ADMIN_USERNAME` / `SEED_ADMIN_PASSWORD` env vars — creates the admin user if missing,

or resets its password if it already exists. Safe to re-run (e.g. if you ever lock yourself out), unlike a one-time-then-delete script. See §16.6 for the concrete version.

9.4 Monitoring & backup

Consolidating identity raises the stakes of both: `syncaxis-iam` being down blocks *new* logins to every app at once (already-signed-in users keep working for up to 8h thanks to the re-verify grace period in §4.3, but no one new can sign in anywhere). This service — and its health check (§8.1) — should be the first thing on your monitoring, ahead of any individual app. `AuthCenter` needs a real backup schedule (SQL Server Express supports scheduled backups via `sqlcmd` /a scheduled task, since Express has no SQL Agent) — losing this DB without a recent backup means resetting every employee’s access across every app simultaneously, not just one app’s worth of accounts as today.

10. Admin UI

Same seven-screen shape as v1 (Dashboard, Users, Roles, Apps & Permissions, Audit Log), with the permission-matrix screen deepened to show actions as columns within each app’s modules, grouped rather than a flat checkbox list:

Role: SalesManager

[Save]

Leads Tracker

	view	create	update	delete	export
Leads	☒	☒	☒	☐	☒
Customers	☒	☐	☒	☐	☐

ERP Dashboard

	view	export
Sales	☒	☒
Finance	☐	☐

Company Portal

	view	update	manage
Employee Directory	☒	☐	—
Admin Console	☐	—	☐

One save updates `RolePermissions` for every user (directly or via a group) holding that role. Server-rendered (EJS/Pug + form posts), as recommended in v1, for the same low-ops-burden reason.

11. Technology Stack

Unchanged from v1: Node.js + Express, `mssql`, `jsonwebtoken`, `bcrypt` / `bcryptjs`, `express-rate-limit`, `helmet`. This matches what Portal and Leads Tracker already use, so the migration in §13 is largely a matter of relocating already-working code, not learning a new stack.

12. Security Considerations

- Password storage: bcrypt, proper cost factor — unchanged from v1.
- Login throttling + account lockout — already implemented in Portal's `auth.js` ; port as-is.
- Access tokens never reach the browser — they live only in each app's backend session store, which removes an entire class of XSS-token-theft concern that a browser-visible JWT would carry.
- App session cookies: `HttpOnly` , `Secure` , `SameSite=Lax` .
- CSRF on state-changing requests: standard cookie-auth practice, per app.
- Audit everything security-relevant: login success/failure, SSO issue/exchange, permission denials, admin changes, forced logouts.
- Least privilege on the `AuthCenter` DB login: no access to `SYNCAXIS` (ERP) or any app's own business DB.
- Secrets (JWT signing key, DB connection string) in environment variables, excluded from version control.
- Operational risk from consolidation (monitoring, backup) is covered in §9.4, not repeated here — but treat it as a security concern, not just an ops one: this DB is now the single thing standing between an outage/data-loss event and every app's access at once.

13. Migration & Rollout Plan

Grounded in what already exists today, not a green-field build:

1. **Stand up `syncaxis-iam` + `AuthCenter`** . Port Portal's existing `auth.js` / `middleware/auth.js` (bcrypt, logout, JWT issuance, role/group resolution) into the new service largely as-is. Add the `Apps / Permissions` self-registration endpoints (§8.2/§8.3) and the `TokenValidAfter` cutoff (§6.4), neither of which exist yet. Bootstrap the first admin per §9.3 before anything else.
2. **Migrate data:** `portal.Users` / `Roles` / `UserGroups` / `RolePermissions` rows move into `AuthCenter` — bcrypt hashes carry over unchanged, no forced password reset. Expand existing `page / application` flags into the new `app.module.action` key convention (§6.1) — this is the one genuinely new modeling work, done once per existing permission. **Before dropping `portal.Users` , add `AuthUserId` to Portal's own `Employees` table and backfill it from the old `portal.Users.EmployeeId` mapping** — otherwise Portal's Directory/org-chart loses the link between a login and an employee record (§7).
3. **Pilot on ERP Dashboard** (lowest visibility, already isolated behind its own SSO-handoff flag): point it at `syncaxis-iam` instead of Portal — same request/response shapes it already expects.
4. **Cut over Leads Tracker:** change its `config.portal.apiUrl` to point at `syncaxis-iam` ; its `auth.ts` / `AuthContext.tsx` / `SsoCallbackPage.tsx` barely change, since they already only talk to an external identity API, never to Portal's business logic. Update its route handlers to check per-action keys (`leads.leads.view/create/update/delete/export`) instead of the current coarse `hasAccess` / `hasAdminAccess` booleans.
5. **Cut over Portal last** (the largest single step): strip its own login route, JWT signing, and `Users` / `Roles` / `UserGroups` admin screens; make Portal consume `syncaxis-iam` the same way Leads Tracker does today — Leads' client code is effectively the reference implementation to copy.

6. **Retire the old code:** drop `portal.Users` etc. from Portal's DB once all three apps are stable on `syncaxis-iam`; delete Portal's now-dead admin screens.
7. **Onboarding any future app** from this point on is just: register it + its permission manifest with `syncaxis-iam` (§8.3), implement the three-endpoint client pattern (§4), done — no step above needs to be repeated.

Keep each app's old auth path available (feature-flagged off) until its migration is confirmed solid, so there's an instant fallback per app during rollout, same principle as v1.

14. Mobile / Native Client Roadmap

Not being built now — no mobile app exists yet — but this is what changes, and what doesn't, when one is. Written now so building a mobile app later doesn't force a redesign of anything already shipped.

14.1 What stays the same

`syncaxis-iam`'s DB schema, admin API, and RBAC/permission model (§6, §7) are untouched by adding mobile. A mobile app is just another registered `App` (§7) with its own permission keys, same as any web app. Web apps (Portal, Leads Tracker, ERP Dashboard, and any future web app) keep exactly the pattern already documented in §4 — nothing about adding mobile forces a change there.

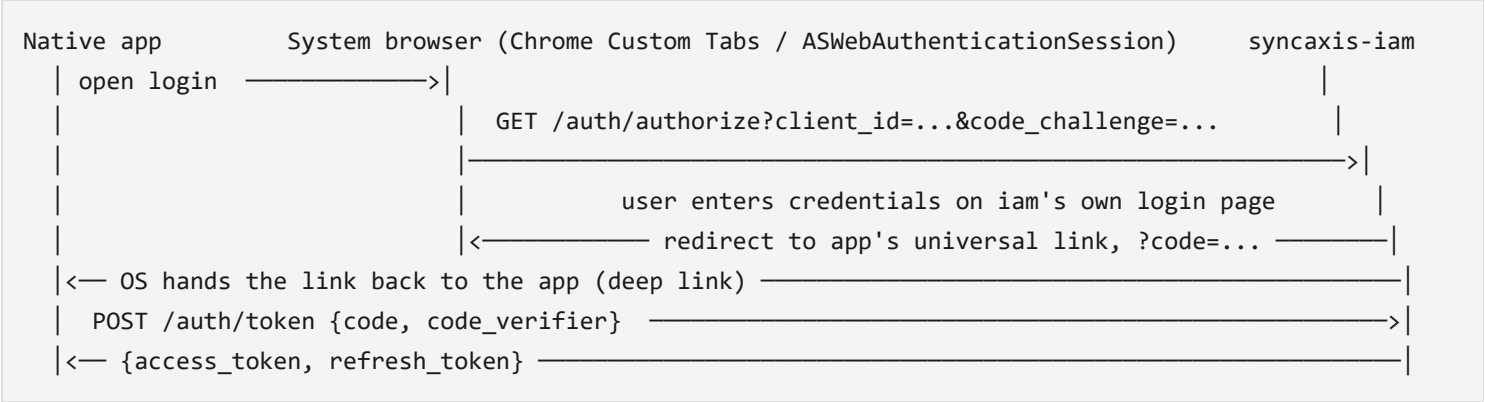
14.2 Why the web pattern doesn't translate directly

Two mechanical gaps: no browser means no `HttpOnly` cookie (a native app needs its own on-device token storage — iOS Keychain / Android Keystore), and no `window.open` means SSO needs a deep link (iOS Universal Links / Android App Links carrying the code) instead of a query-string parameter on a browser tab.

The more important gap is trust, not mechanics: a native app is a **public client** — it ships to a user's device and can be decompiled, unlike Portal's or Leads' backends, which never leave your server room. Lifting §4.1's login-proxy pattern (app relays a raw password server-to-server) unmodified onto a phone app would mean the app itself becomes trusted to see and relay a raw password — something you have no ongoing control over once it's installed on hundreds of phones. Mobile needs a pattern designed for that, not a workaround.

14.3 The pattern to use: OAuth2 Authorization Code + PKCE

This is the same standards-based fallback already named in §15 (Trade-offs, now folded into this section) as the eventual outgrow-path — a native app is the concrete reason to build it, not a hypothetical scale trigger.



Key differences from §4’s web pattern: - Credentials are entered on `syncaxis-iam`’s own login page, rendered inside a system-controlled browser component (Chrome Custom Tabs / `ASWebAuthenticationSession`) — never an in-app `WebView` the app’s own code could read from. This is the OS-vendor-recommended pattern specifically because it means a raw password never passes through app code at all, public client or not. - PKCE (`code_challenge` / `code_verifier`) is what lets a public client (no way to hold a secret) prove it’s the same app that started the flow — the mobile equivalent of what a trusted backend gives web apps for free. - The native app itself holds the resulting tokens in OS secure storage — there’s no backend of yours in between.

14.4 Token lifetime for mobile — the one real schema addition, deferred until needed

§4.4’s 8-hour access token with no refresh is fine for a web session but wrong for a phone app people expect to stay signed into for weeks without re-entering a password. A native client needs:

Token	Lifetime	Stored
Access token	15–30 min	In-memory / short-lived on-device
Refresh token	30–90 days, rotated on use	OS secure storage (Keychain/Keystore), hashed at rest server-side

This reintroduces a `RefreshTokens` table (`TokenId` , `UserId` , `TokenHash` , `ExpiresAt` , `RevokedAt`) — deliberately dropped from the v2 schema (§0) because no client needed it yet, not because it can’t come back. Scope it to native clients only, so web apps stay on today’s simpler model; add the table when the first mobile app is actually built, not speculatively now.

14.5 What `Apps` needs when this is built

One column: `Apps.ClientType` (`'web-backend'` | `'native-public'`), so `syncaxis-iam` knows which flow (§4 vs §14.3) and which token-lifetime rules (§4.4 vs §14.4) apply to a given registered app. Everything else in §7’s schema carries over unchanged — this is additive, not a migration of existing data.

15. What to revisit as this grows

- **Row-level scoping or field-level masking** (§6.5) — layer on only if a concrete role actually needs “own records only” or hidden fields; not needed today.
 - **A native mobile app** — see §14; the trigger to actually build the PKCE flow and `RefreshTokens` table.
 - **More than ~15–20 apps, or truly external/partner access**: the same PKCE-based flow built for mobile (§14.3) covers this too — one standards-based flow serves both drivers.
 - **Need for MFA**: add TOTP at the `/auth/login` (web) or `/auth/authorize` (mobile) step; nothing else in this document changes.
 - **Compliance requirements** (e.g. a customer demands SSO audit evidence): this is where adopting an OIDC-compliant layer (Keycloak or similar) earns its added complexity — still a drop-in replacement for `syncaxis-iam`’s auth endpoints, since every app talks to it only through the contract in §8.
-

16. Appendix: Implementation Specification

Everything above this line is architecture. Everything below is the concrete spec needed to generate the service without further back-and-forth — pinned choices, not open questions.

16.1 Language & package versions

TypeScript (decided over plain JS to match Portal’s style) — the single point of trust for every app’s login/permissions is worth the extra safety of typed request/response shapes, and it matches half the existing stack (Leads Tracker) already.

Pinned to match what’s already proven in production on this exact Windows Server + SQL Server Express setup (Portal’s and Leads’ own `package.json`), not newer/unproven majors:

```
{
  "dependencies": {
    "express": "^4.19.2",
    "mssql": "^11.0.1",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "dotenv": "^16.4.5",
    "helmet": "^7.1.0",
    "express-rate-limit": "^7.4.0",
    "ejs": "^3.1.10"
  },
  "devDependencies": {
    "typescript": "^5.4.5",
    "ts-node-dev": "^2.0.0",
    "@types/express": "^4.17.21",
    "@types/mssql": "^9.1.5",
    "@types/bcryptjs": "^2.4.6",
    "@types/jsonwebtoken": "^9.0.6",
    "@types/node": "^20.14.2"
  }
}
```

Node 20 LTS (matches Leads Tracker's `@types/node ^20`). No `cors` package — the API is server-to-server only (§8.1), and the admin UI (§10) is server-rendered by this same Express app, so nothing ever calls it cross-origin.

16.2 Folder structure

Mirrors Portal's existing layout (which this service's auth code is largely ported from), adapted for TypeScript per Leads' layout:

```

syncaxis-iam/
src/
  config/
    env.ts          # reads + validates all env vars (§16.3), throws on startup if any required one is
    db.ts           # mssql connection pool (getPool())
  middleware/
    auth.ts         # requireAuth (bearer token), requireAdmin, requirePermission(key)
  routes/
    auth.ts         # POST /auth/login, /auth/sso/issue, /auth/sso/exchange, GET /auth/me, POST /aut
    admin/
      apps.ts       # POST /admin/apps, POST /admin/apps/:key/permissions
      users.ts      # GET/POST /admin/users, POST /admin/users/:id/roles|groups|force-logout
      roles.ts      # GET/POST /admin/roles, POST /admin/roles/:id/permissions
      audit.ts      # GET /admin/audit
      health.ts     # GET /health
  admin-ui/
    views/          # EJS templates – dashboard, users, roles, apps, audit (§10)
    views-routes.ts # renders the views, posts to the JSON admin API above
  lib/
    audit.ts        # writeAuditLog(event) helper, one place every route calls into
    permissions.ts  # getEffectiveAccess(pool, userId) – ported from Portal's middleware/auth
    app.ts          # express() wiring: helmet, rate-limit, json body parser, mount routes,
    server.ts       # entrypoint: connect DB, app.listen()
  scripts/
    seed.ts         # bootstrap/reset the first admin (§9.3) – ported from Portal's seed.js
  .env.example
  package.json
  tsconfig.json

```

16.3 Environment variables

```
PORT=4100
DB_SERVER=192.168.3.9
DB_INSTANCE=SQLEXPRESS
DB_NAME=SYNCAXIS_AUTHCENTER
DB_USER=syncaxisadmin
DB_PASSWORD=
DB_ENCRYPT=false
DB_TRUST_SERVER_CERTIFICATE=true

JWT_SECRET=
JWT_EXPIRES_IN=8h

SESSION_TTL_HOURS=8
REVERIFY_INTERVAL_MINUTES=5
SSO_CODE_TTL_SECONDS=60

# Login throttling (§12) – mirrors Portal's existing MAX_FAILED_LOGINS=3 pattern
MAX_FAILED_LOGINS=3

# Bootstrap script only (§9.3) – never read outside `npm run seed`
SEED_ADMIN_USERNAME=
SEED_ADMIN_PASSWORD=
```

`config/env.ts` throws on startup if `DB_SERVER` / `DB_NAME` / `DB_USER` / `DB_PASSWORD` / `JWT_SECRET` are unset — same fail-fast pattern as Portal's `config/env.js`.

16.4 API schemas

POST /auth/login Request: { "username": string, "password": string } Success 200: { "token": string, "user": UserSummary } Error 400: { "error": "Username and password are required." } Error 401: { "error": "Incorrect username or password." } Error 423: { "error": "This account is locked. Contact an administrator to unlock it." }

UserSummary (returned by `/auth/login`, `/auth/sso/exchange`, `/auth/me`):

```
{
  id: number
  username: string
  displayName: string
  isActive: boolean
  roles: { id: number; name: string }[]
  perms: string[] // flat, deduplicated list of every permission key granted,
                  // e.g. ["leads.leads.view", "leads.leads.update",
                  // "erp.dashboard.sales.view"]
  isFullAccess: boolean // true if any granted role has IsFullAccess – apps should treat this as
                        // "all perms"
  lastLoginAt: string | null // ISO 8601
}
```

POST /auth/sso/issue — requires `Authorization: Bearer <token>` from the *issuing* app's own established session
 token Success 200: { "code": string } Error 401: { "error": "Not authenticated." }

POST /auth/sso/exchange Request: { "code": string } Success 200: { "token": string, "user": UserSummary }
 Error 401: { "error": "This sign-in link has expired - please try again from the Portal." }

GET /auth/me — requires `Authorization: Bearer <token>` Success 200: { "user": UserSummary } Error 401: {
 "error": "Session expired or invalid – please sign in again." }

POST /auth/logout Success 204, no body.

GET /health Success 200: { "ok": true, "db": "connected" } — pings the DB pool; 503 with { "ok": false } if the ping fails.

All admin endpoints (§8.2) follow the same {error: string} convention on failure and require `Authorization: Bearer <token>` plus `iam.admin.manage` (checked via `requirePermission`).

16.5 Validation rules

- Username: required, trimmed, 3–100 chars, unique.
- Email: required, unique, standard email format check.
- Password (on create/reset): minimum 8 characters — matches Portal's existing `change-password` rule.
- `MAX_FAILED_LOGINS` (default 3): on the 3rd consecutive failure, set `IsLocked = 1`; unlocking is an explicit admin action (`POST /admin/users/:id` with `isLocked: false`), which also resets `FailedLoginCount` .
- Permission key format: must match `^[a-z][a-z0-9]*(\.[a-z][a-z0-9-]*)+$` and must start with the calling app's own `Key` + `.` (§8.3 namespace enforcement).

16.6 Bootstrap script (`scripts/seed.ts`)

Behavior, ported from Portal's proven `scripts/seed.js` :

1. Read `SEED_ADMIN_USERNAME` / `SEED_ADMIN_PASSWORD` from env; if either is unset, log and exit without error (safe to run in an environment that doesn't intend to seed).

2. `bcryptjs.hash(password, 12)` .
3. If a user with that username exists: update its `PasswordHash` , set `IsActive = 1` , bump `TokenValidAfter = SYSUTCDATETIME()` (so any old session is invalidated by the reset).
4. Else: insert the user, then insert a `UserRoles` row linking it to the seeded `Admin` role (`IsFullAccess = 1`).
5. Idempotent and safe to re-run at any time — this is the account-recovery path if you ever get locked out, not a one-shot install step.

16.7 Audit log conventions

`EventType` values and what `Detail` carries for each, so every route logs consistently:

EventType	Detail contains
<code>LOGIN_SUCCESS</code> / <code>LOGIN_FAILURE</code>	username attempted
<code>ACCOUNT_LOCKED</code>	username, failed attempt count
<code>SSO_ISSUE</code> / <code>SSO_EXCHANGE</code>	issuing/destination app key
<code>PERMISSION_DENIED</code>	permission key that was required, route path
<code>ROLE_CHANGED</code> / <code>GROUP_CHANGED</code>	target user id, role/group id, added or removed
<code>FORCE_LOGOUT</code>	target user id, admin user id who triggered it
<code>APP_REGISTERED</code> / <code>PERMISSIONS_UPDATED</code>	app key, count of permission keys declared

`AppKey` column is set whenever the event relates to one specific consuming app (SSO, permission denial); `NULL` for purely admin/user-management events.